

Git: A Time Machine for Your Files

1 The Problem: Which Version Is Correct?

Imagine you are working on a school project in Microsoft Word. You start with a file called:

```
Science_Project.docx
```

You make some changes and save it. A few days later, you make more changes. Then you realize that the older version was actually better. So you start making copies:

```
Science_Project.docx
Science_Project_Final.docx
Science_Project_Final2.docx
Science_Project_Final_Updated.docx
```

Sound familiar? Now imagine that three students are working on the same document.

Ali changes the introduction:

```
Science_Project_Ali.docx
```

Sara fixes the introduction:

```
Science_Project_Sara.docx
```

Ahmed adds his changes:

```
Science_Project_Ahmed.docx
```

Now you have a problem:

- Which file is the correct one?
- What exactly did each person change?
- Can we go back to the version we had yesterday?

2 What If We Had a Time Machine?

Imagine that instead of creating dozens of copies, we had a time machine for our project.

At 10:00 AM:

Version 1
“Initial project”

At 11:30 AM:

Version 2
“Added introduction”

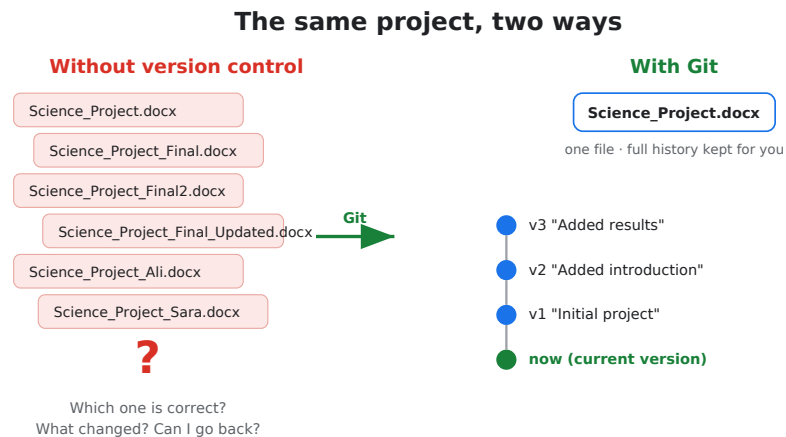
At 2:00 PM:

Version 3
“Added results”

Every important point in the project's history is saved. We can look at the history and see:

- what versions exist,
- when they were saved,
- what changed between versions,
- and, when needed, return to an earlier version.

This is the basic idea behind **version control**. Version control keeps a history of your files so you can understand and manage changes over time.



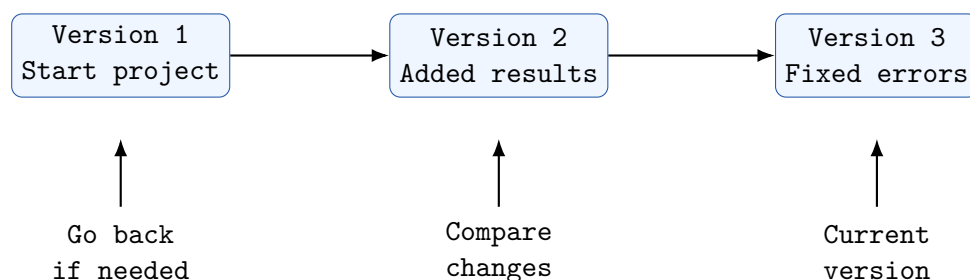
3 Git: Our Time Machine

Git is a tool for version control. Instead of manually creating files such as:

```
Final.docx
Final2.docx
Final_NEW.docx
```

we let Git keep track of the history for us. Think of Git as a time machine for your project.

YOUR PROJECT



Each saved point is called a **commit**. For now, think of a commit as a saved version of your project.

For example:

```
Commit 1 -> "Create project"
Commit 2 -> "Add introduction"
Commit 3 -> "Add results"
Commit 4 -> "Fix calculation"
```

Git remembers these versions for us. You don't need to remember every change yourself. You tell Git:

“Save this version.”

Git remembers it.

Later, you can ask:

- “What did I change?”
- “What versions do I have?”
- “What was my project like before I made this change?”

That is why we can think of Git as a time machine for our project.

The bonus point is that Git works for many types of files, including programming/code files such as `.py`, `.c`, and `.cpp`, text files, documents, images, and more.

4 Creating Our First Git Repository

Now that we understand the problem, let's create our own Git time machine.

We will use a small project called **MyCalculator**.

4.1 Step 1: Create a Project Folder

Create a folder called:

```
MyCalculator
```

Inside it, create a simple file:

```
calculator.py
```

You can put 2–3 lines of simple Python code in the file, for example, a few `print()` statements.

Your folder should look like this:

```
MyCalculator/
calculator.py
```

The folder contains our project. We want Git to keep track of what happens inside this folder.

4.2 Step 2: Tell Git to Track the Project

Open a terminal inside the **MyCalculator** folder and type:

```
git init
```

You should see a message similar to:

```
Initialized empty Git repository...
```

What just happened?

We told Git:

“Start keeping track of this project.”

The folder is now a Git repository.

What is a repository?

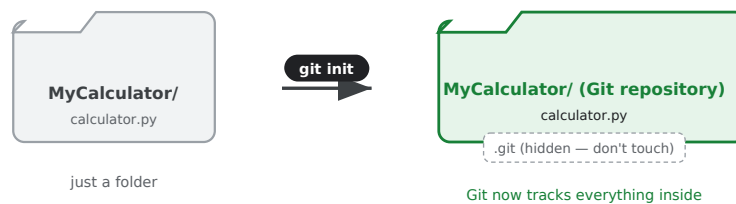
A repository, or **repo**, is simply a project folder that Git is tracking.

Normal folder → `git init` → **Git repository**

Git also creates a hidden `.git` folder inside the project. This is where Git stores the information it needs to maintain the project’s history.

Do not modify or delete the `.git` folder.

git init turns a normal folder into a repository



4.3 Step 3: Ask Git What Is Happening

Now type:

```
git status
```

Git will tell us about the current state of our project.

You may see something similar to:

```
Untracked files:
  calculator.py
```

This is what the same command will look like for you on your command line interface (CLI)

Git has noticed our file, but there is an important point:

Git knows the file exists, but we have not yet told Git to include it in a saved version.

This is what **untracked** means.

Think of it like this:

```

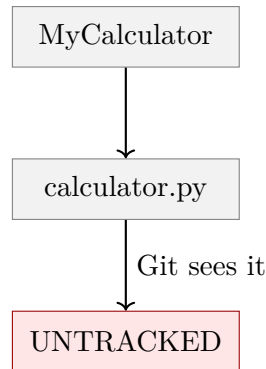
PS C:\Users\sumaira.khan\OneDrive - Habib University\Desktop\My_project> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .micropico
        .vscode/
        blink.py

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\sumaira.khan\OneDrive - Habib University\Desktop\My_project>

```



We are now ready to make our first saved version.

RA Checkpoint

Checkpoint 1 — RA Approval: Repository Setup

Show your RA:

- The MyCalculator folder containing calculator.py.
- The repository initialized using `git init`.
- The output of `git status` showing calculator.py as an untracked file.

RA Approval: Confirm that the project has been correctly initialized as a Git repository and that the student understands the meaning of an untracked file.

Student Names: _____ **RA Initials:** _____

5 A Quick Check

At this point, remember these important terms:

Term	Meaning
Git	A tool that keeps the history of our project
Repository	A folder that Git is tracking
git init	A command that initializes a repository
git status	Shows the current state of our project

Our first Git workflow:

Create project → `git init` → **Git starts tracking the project** → `git status` → **See what Git knows about**

6 Making Our First Commit: Saving a Version

We have created our Git repository, and Git can see `calculator.py`. But there is a problem. Git knows that `calculator.py` exists, but we have not saved it as a version yet. Remember our time-machine idea? We want to create our first save point:

Version 1
“Create calculator”

To do this, we use two commands:

```
git add calculator.py
git commit -m "Create calculator"
```

Tip: `git add -A` adds all the files in a folder instead of adding them one by one.

Let’s understand them one at a time.

6.1 Step 1: Prepare the File with `git add`

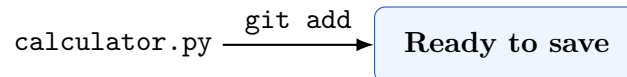
Type:

```
git add calculator.py
```

The `git add` command tells Git:

“I want to include this file in my next saved version.”

Think of it like putting a document into a box before sealing the box.



You can check the situation with:

```
git status
```

You should now see something similar to:

```
Changes to be committed:
  new file:   calculator.py
```

The important thing to notice is:

`git add` does not save a version.

It prepares the changes that we want to save.

6.2 Step 2: Save the Version with `git commit`

Now type:

```
git commit -m "Create calculator"
```

The `git commit` command tells Git:

“Save the prepared changes as a new version.”

The `-m` allows us to give this version a message.

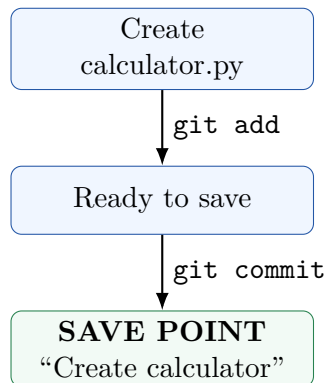
Our message is:

```
"Create calculator"
```

So our first commit is:

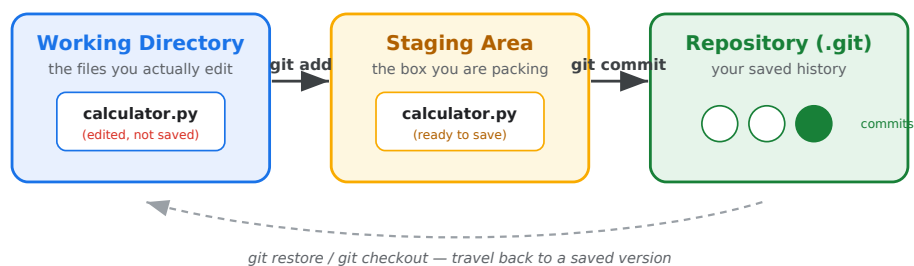
Commit 1
"Create calculator"

Think of the commit as pressing the **Save Point** button on our time machine.

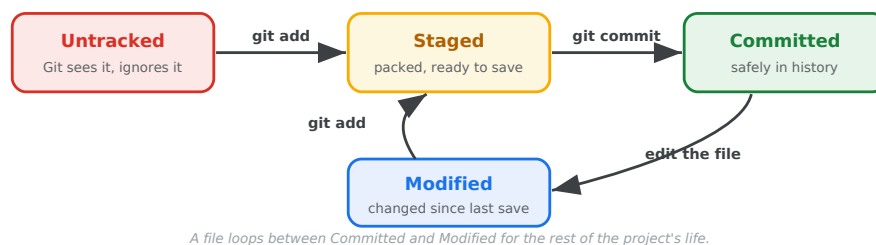


This saved point is called a **commit**.

The three places your file can live



The life of a file (what git status is telling you)



6.3 Step 3: Check the Repository Again

Now run:

```
git status
```

You should see something similar to:

```
nothing to commit, working tree clean
```

This is good news!

It means there are no new changes waiting to be saved. Our project is now at a saved point.

RA Checkpoint

Checkpoint 2 — RA Approval: First Commit

Show your RA:

- The `git add calculator.py` command.
- The `git commit -m "Create calculator"` command and its result.
- The output of `git status` showing a clean working tree.

RA Approval: Confirm that the student has successfully created the first commit and understands the difference between `git add` (prepare changes) and `git commit` (save a version).

Student Names: _____ **RA Initials:** _____

7 Why Do We Need a Commit Message?

Imagine that you make 20 commits but give every commit the message:

```
Changes
```

After a few days, your history might look like:

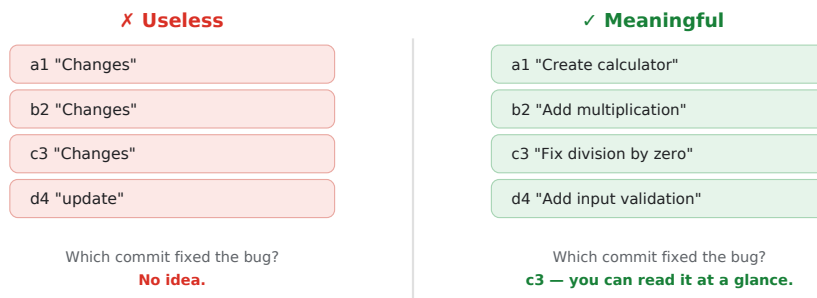
```
Changes
Changes
Changes
```

That isn't very helpful.

Instead, write a short message that explains what you changed:

```
Create calculator
Add multiplication
Fix division
Add input validation
Improve error message
```

Why commit messages matter



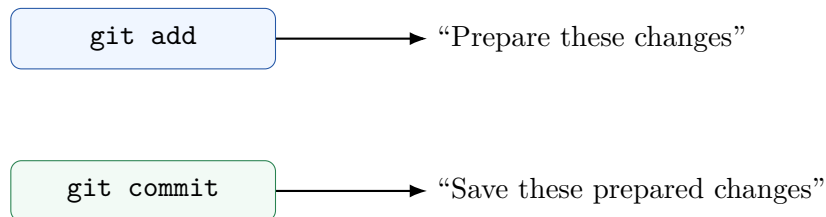
A good commit message should answer:

“What did I change in this version?”

Keep it short and meaningful.

8 The Important Difference: add vs commit

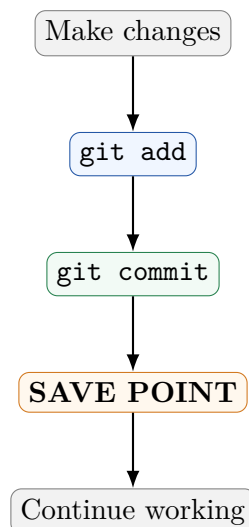
Students often confuse these two commands, so remember:



Or, using our time-machine analogy:

`git add` → Choose what goes into the save point
`git commit` → Create the save point

So our basic workflow is now:



9 One Important Habit

Commit when you have reached a meaningful point in your work.

For example:

```
"Create calculator"
"Add multiplication"
"Add division"
"Fix input error"
```

You don't need to create a commit after every single character you type.

Think of a commit as:

“This is a useful version of my project that I may want to remember.”

10 Looking Back: Our Git History

We have made our first commit.

But the real power of our time machine is that Git remembers our saved versions.

Let's see what Git remembers.

Type:

```
git log
```

You should see something similar to:

```
commit a82f31...
Author: Ali
Date:   ...

    Create calculator
```

This is our project's history.

Every time we make a commit, Git adds another entry to this history.

For example, after making a few more commits, our history might look like:

```
commit c31a52...
    Add division

commit b72f10...
    Add multiplication

commit a82f31...
    Create calculator
```

Notice that the latest commit appears first.

Each commit also has a long-looking ID, such as:

```
a82f31...
```

This is how Git identifies that particular commit.

For now, you don't need to understand how these IDs are created. Just remember:

Every commit has its own ID.

The normal `git log` contains quite a lot of information.

For a quick look at our history, use:

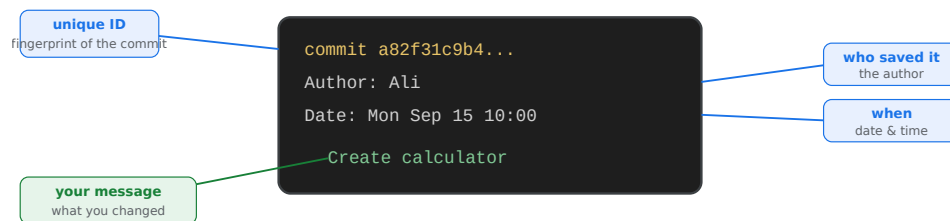
```
git log --oneline
```

You might see:

```
c31a52 Add division
b72f10 Add multiplication
a82f31 Create calculator
```

This is often easier to read.

How to read one entry from git log



Tip: `git log --oneline` shows just the ID + message, one per line.

11 What Did I Change?

The history tells us which versions exist, but sometimes we want to know something more specific:

“What did I change after my last commit?”

This is where `git diff` helps.

Let's try it.

First, open `calculator.py` and make a small change.

For example, imagine you had a print line in your file. Change:

```
print("Hello")
```

to:

```
print("Hello, Ali!")
```

Do not commit the change yet.

Now type:

```
git diff
```

Git will show us the difference between our last saved version and our current file.

You may see something similar to:

```
- print("Hello")
+ print("Hello, Ali!")
```

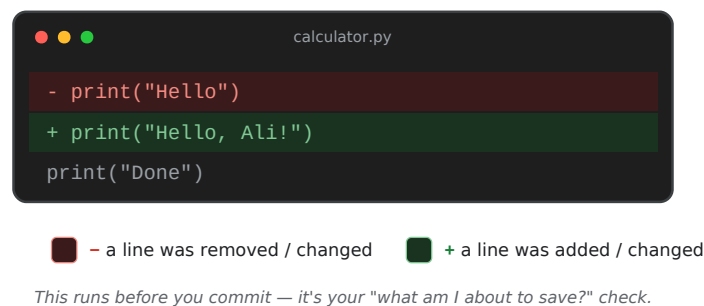
The symbols are simple:

- - → something was removed or changed
- + → something was added or changed

So `git diff` answers:

“What have I changed since my last commit?”

git diff shows what changed since your last commit



12 The Complete Git Cycle

We now know the basic commands needed to work with Git locally.

Imagine you are working on your calculator project.

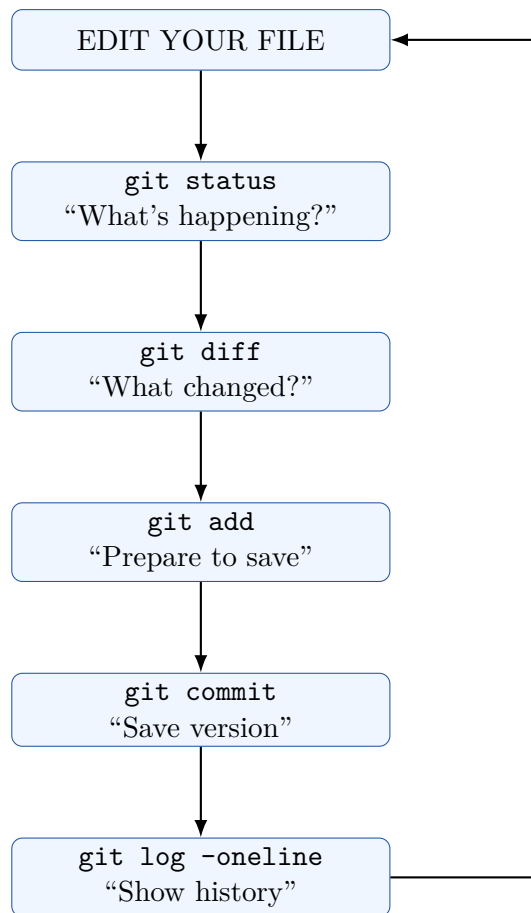
You make some changes.

First, you can ask:

What is happening?	<code>git status</code>
What exactly did I change?	<code>git diff</code>
Prepare my changes	<code>git add .</code>
Save this as a new version	<code>git commit -m "Add multiplication"</code>
Show my saved versions	<code>git log --oneline</code>

The dot in `git add .` is part of the command.

Put together:



This is the basic Git workflow.

RA Checkpoint

Checkpoint 3 — RA Approval: Git History and Changes

Show your RA:

- The output of `git log -oneline` showing the project’s commit history.
- At least three meaningful commits with descriptive commit messages.
- A modified `calculator.py` file.
- The output of `git diff` showing the changes that have not yet been committed.

RA Approval: Confirm that the student can view the project’s history using `git log` and identify current uncommitted changes using `git diff`.

Student Names: _____ **RA Initials:** _____

13 Your Git Time Machine

After several commits, your project might have a history like this:

Add input validation — newest
Add division
Add multiplication
Create calculator — oldest

Each commit is a saved point in time.

Your current files are your work right now.

Your commits are the versions you have chosen to remember.

And `git diff` lets you see the changes between your current work and the last saved point.

That is the basic idea of Git:

**Work → See your changes → Save a meaningful version →
Continue working**

You are building a history of your project as you work.

14 Git Cheat Sheet

What do I want to do?	Command
Start Git in my project	<code>git init</code>
See what is happening	<code>git status</code>
See what I changed	<code>git diff</code>
Prepare changes to save	<code>git add .</code>
Save a new version	<code>git commit -m "message"</code>
See my project history	<code>git log -oneline</code>

15 The Golden Rule

Don't think of Git as a collection of commands to memorize.

Think of it as a conversation with your project:

```

    "What is happening?" → git status
    "What did I change?" → git diff
    "Save this version." → git add + git commit
    "What have I saved?" → git log

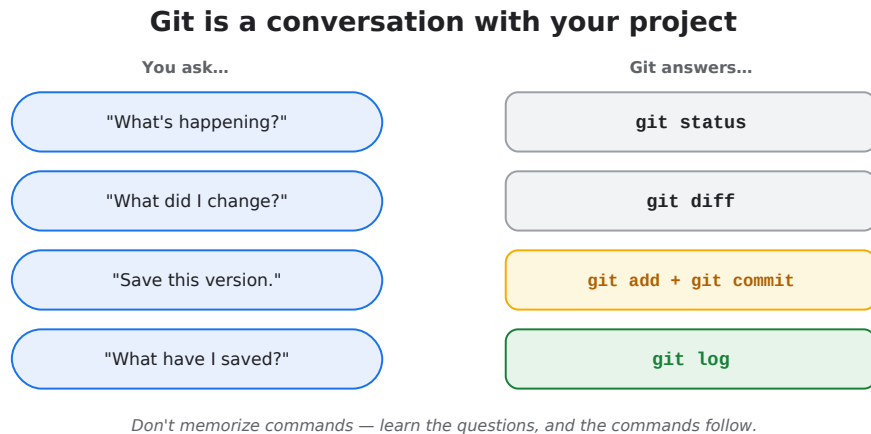
```

And that is enough to get started with Git :-)

16 Looking Ahead: When You Are Not Working Alone

Remember how we started? Three students were working on the same project:

- Ali changed the introduction,
- Sara fixed the introduction,
- Ahmed added his changes.



And we asked three questions: *Which file is correct? What did each person change? Can we go back to an earlier version?*

Everything in this tutorial, `git commit`, `git log`, `git diff`, going back — answers those questions for **one person working alone**. Your project now has a personal time machine.

But look closely: Ali, Sara, and Ahmed each built their *own* time machine on their *own* laptop. That doesn't help them work together yet. If Ali makes a commit on his laptop, Sara has no way to see it. So one question from the very beginning is still open:

How do several people share one history?

That is the next chapter, and the tool is **GitHub**.

GitHub is a place on the internet where a Git repository can live so that everyone can reach it. Think of it as **one shared time machine in the cloud** that Ali, Sara, and Ahmed all connect to.

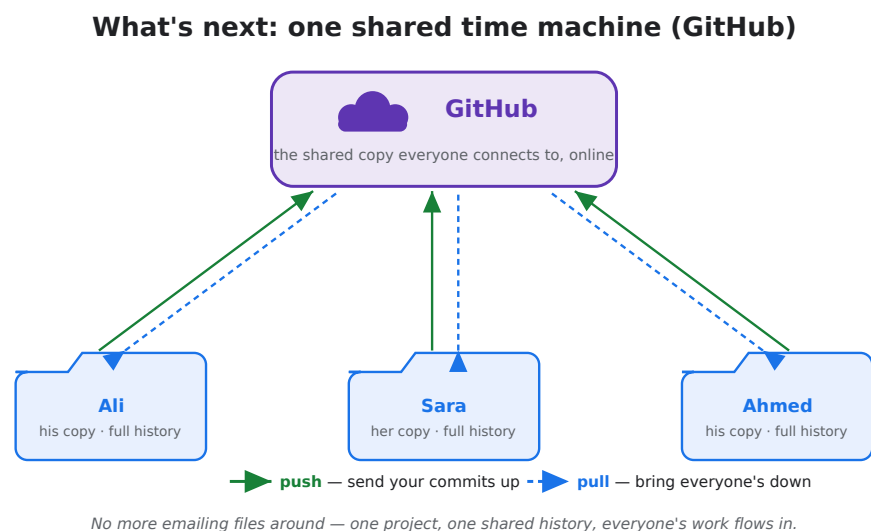


Figure 1: One shared history: everyone *pushes* their commits up to the same GitHub repository and *pulls* everyone else's down.

Here is the idea, not the full mechanics yet:

- Everyone keeps their own copy of the project, with the full history, on their own machine,

exactly the repository you built in this tutorial.

- Everyone connects that copy to the same shared repository on GitHub.
- When you finish some work, you send your commits **up** to GitHub. This is called a **push**.
- When you want everyone else's latest work, you bring their commits **down**. This is called a **pull**.
- If two people changed *different* things, Git combines them for you automatically (a **merge**). If they changed the *same* line, Git stops and asks a human to decide (a **merge conflict**).

So the messy folder of `Science_Project_Ali.docx`, `Science_Project_Sara.docx`, and `Science_Project_Ahmed.docx` simply disappears. There is one project, one shared history, and everyone's changes flow into it.

A quick look ahead (you don't need these yet, just meet the words):

Term	Meaning (for now)
GitHub	A website that hosts your Git repository so others can reach it
clone	Make your own copy of a project that already exists on GitHub
push	Send your commits up to GitHub
pull	Bring other people's commits down to your copy
merge	Combine changes from different people into one history

The one idea to hold on to:

- **Git** (this tutorial) = save and manage your project's history *on your own machine*.
- **GitHub** (next) = *share* that history so a whole team can work on it together.

We'll pick up Ali, Sara, and Ahmed's story there. :-)